Reflection Template

Use Case	UC-5
Use Case Name	View Project
Requirement IDs	FR5, FR20, FR22, FR23
Matching Feature Comparison Sets	CS-11, CS-12
Implementation Order	Java first -> Kotlin second

Feature Relevance

Feature/s:

Extension functions, Type inferences

Did the feature/s occur naturally in this/these context/s?

Yes, RepositoryUtils was introduced in Java as the equivalent to Kotlin's Extensions.kt, this is the first use case where both are directly compared. The .map{} in Kotlin came up naturally when listing projects, since Kotlin collections don't need the Stream API.

Qualitative Ratings (1-5)

- 1 = very poor/very difficult
- 2 = rather poor/difficult
- 3 = neutral/moderate
- 4 = good/easy
- 5 = very good/very easy

Criteria	Java	Kotlin
Readability	4	5
Compactness of logic	3	5
Perceived implementation effort	4	5

Justifications

Readability:

Both are readable but Kotlin's .map{} is more natural than Java's .stream().map().toList(). The RepositoryUtils.findByIdOrThrow(userRepository, ownerId, "User") syntax in Java is slightly more verbose than Kotlin's userRepository.findByIdOrThrow(ownerId, "User"), the repository is passed as a parameter in Java, while in Kotlin it reads like a native method call.

Compactness of logic:

Kotlin is more compact on two fronts the extension function syntax and the collection mapping. Java needs .stream().map().toList() where Kotlin just uses .map{}.

Implementation effort:

Needed to implement a new RepositoryUtils class with the correct generics. With kotlin just reused what was already there.

Observed structural differences

First time where RepositoryUtils and Extensions.kt can be directly compared. The key difference: in Java the repository is passed as a parameter (RepositoryUtils.findByIdOrThrow(repository, id, "Entity")), in Kotlin the extension function is called on the repository itself (repository.findByIdOrThrow(id, "Entity")). Kotlin reads more naturally.

Kotlin uses .map{} directly on collections, no Stream API needed. Java requires .stream().map().toList() for the same operation.

Earlier Java use cases used orElseThrow inline; did not modify retroactively to preserve the natural implementation progression.

Bias/validity note:

RepositoryUtils were planned from the beginning as the Java equivalent of Kotlin's extension functions but were accidentally forgotten in UC1—UC4. they were introduced in UC5 without modifying earlier use cases. This should be considered when comparing extension function usage across all use cases.

Final comparative summary

first time where the extension function comparison becomes fully visible. The difference in call syntax, passing the repository as a parameter in Java vs. calling the method on it in Kotlin, clearly shows what extension functions add in terms of readability. The collection mapping difference is a small but consistent advantage for Kotlin that will repeat in every future list operation.